Redes Neurais Informadas por Física

Tópicos Especiais em Transferência de Calor

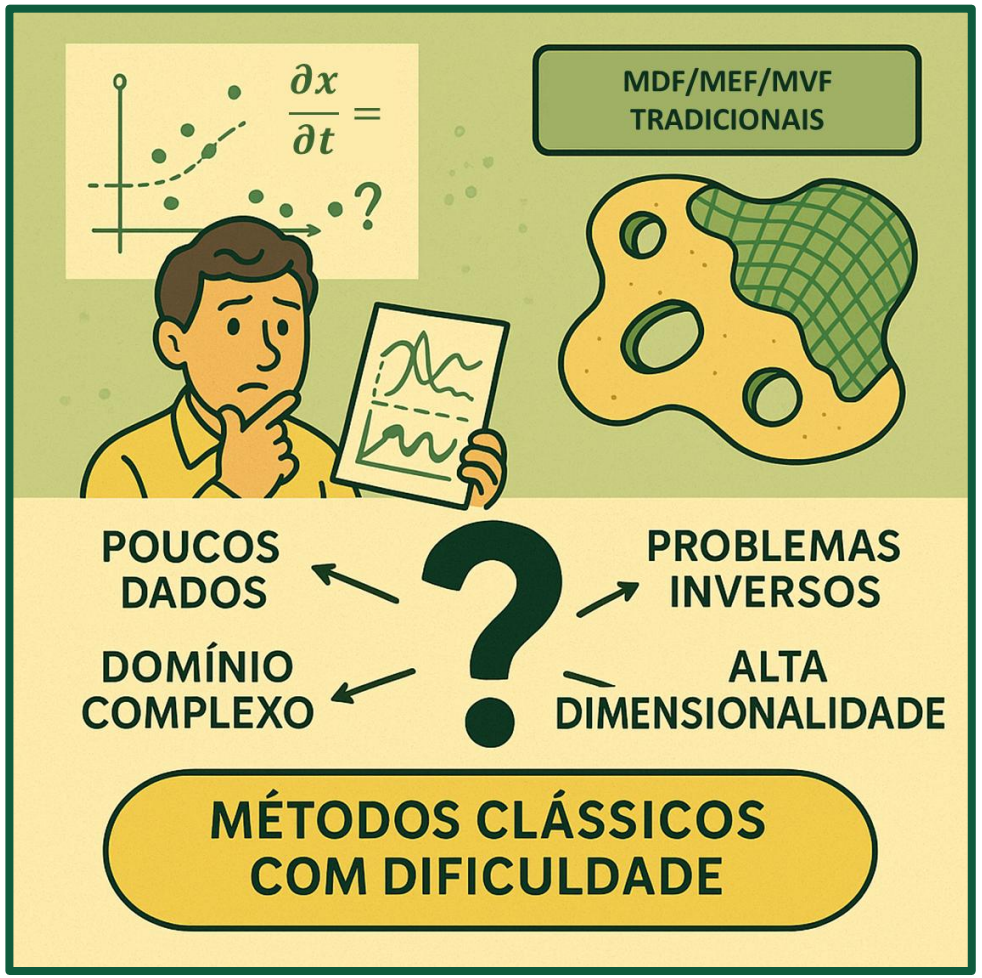
MSc. Eng. Paulo de Souza Silva e Prof. Dr. Daniel Dezan

Novembro de 2026



INTRODUÇÃO E MOTIVAÇÃO

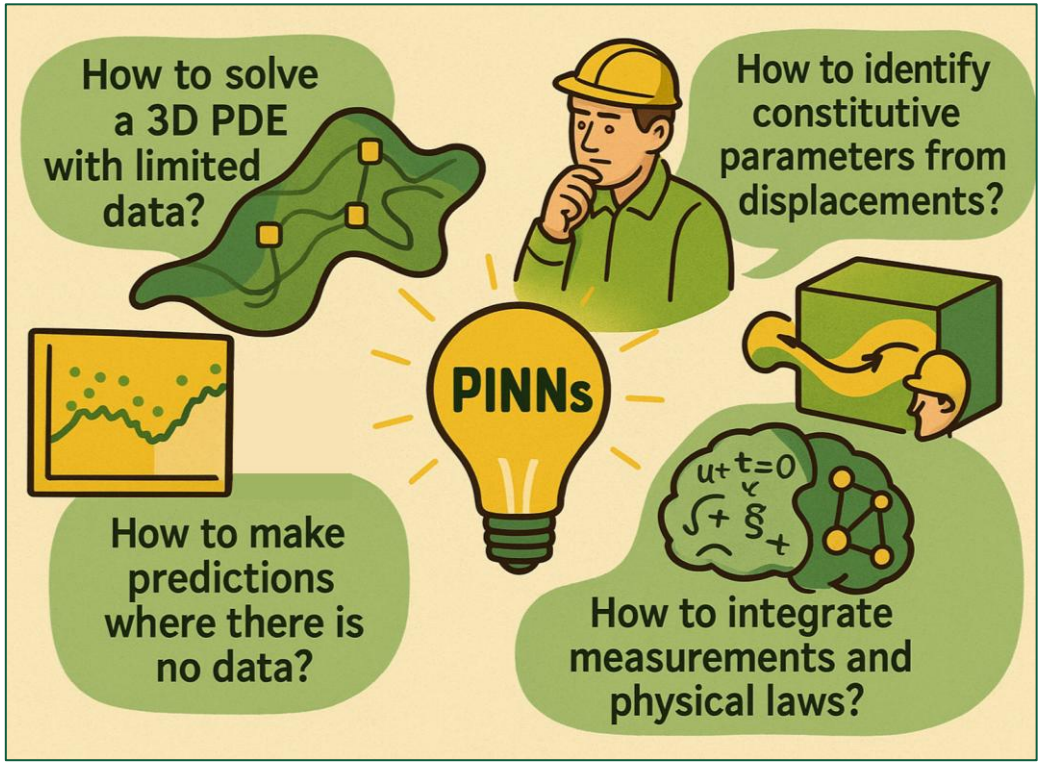
- Leis físicas conhecidas, mas os **dados experimentais** costumam ser **poucos**, **caros** ou **com ruído**.
- Muitas vezes queremos descobrir algo **"escondido"** (**problemas inversos**), como propriedades de um material sem medi-las diretamente.
- **Problemas mal definidos** (mal-postos) aparecem com frequência e **são difíceis** de resolver com **métodos clássicos**.
- Métodos tradicionais, como **elementos finitos**, precisam de **malhas detalhadas**, que são difíceis de criar em geometrias complicadas.
- Quando o problema tem **muitas variáveis ou dimensões**, os métodos numéricos tradicionais começam a falhar.
- É **difícil combinar dados reais** com **simulações clássicas**, porque os métodos numéricos não lidam bem com **medições incompletas** ou **irregulares**.



INTRODUÇÃO E MOTIVAÇÃO

Diante disso, surgem perguntas reais da engenharia e da ciência moderna:

- Como resolver uma EDP em um domínio 3D irregular com poucos dados experimentais?
- Como identificar parâmetros constitutivos de um material só a partir de medidas de deslocamento?
- Como fazer previsões confiáveis mesmo em regiões onde não há dados?
- Como integrar dados medidos e leis físicas em um mesmo modelo?



Essas são as lacunas que motivaram o surgimento das PINNs - Redes Neurais Informadas pela Física



Revisão Inicial

Equações Diferenciais

Redes Neurais

Uma breve ideia das EDOs e EDPs

Equações Diferenciais Ordinárias (EDOs) e Parciais (EDPs)

EDOs: envolvem derivadas em relação a uma única variável independente. $\frac{dy}{dx} = f(x, y)$

EDPs: envolvem derivadas parciais com relação a duas ou mais variáveis independentes. $\frac{\partial u}{\partial t} = k\frac{\partial^2 u}{\partial x^2}$

Classificação

Linear: a função incógnita e suas derivadas aparecem apenas na primeira potência e não multiplicadas entre si. Ex.: $y'' + p(x)y' + q(x)y = g(x)$

Não-linear: contém termos com potências, produtos ou funções não-lineares da incógnita ou de suas derivadas.

Homogênea: se o termo independente for zero. Ex.: $y''+p(x)y'+q(x)y=0$

Não-homogênea: possui um termo independente não nulo. Ex.: $y'' + p(x)y' + q(x)y = g(x)$, com $g(x) \neq 0$

Uma breve ideia das EDOs e EDPs

Problema de Valor Inicial

Problema onde se busca uma solução para uma EDO ou EDP com condições especificadas em um ponto ou superfície inicial.

Exemplo: $\frac{dy}{dx} = f(x, y)$ com $y(x_0) = y_0$

Condições de Contorno: Neumann e Dirichlet

Dirichlet: especifica o valor da **função** na fronteira do domínio. Ex.: $u = g$ em $\partial\Omega$.

Neumann: especifica o valor da **derivada normal** da função na fronteira (fluxo). Ex.: $\frac{\partial u}{\partial n} = h$ em $\partial\Omega$, onde $\frac{\partial}{\partial n}$ indica a derivada na direção normal à fronteira.

Uma breve ideia das EDOs e EDPs

Operador Nabla: ∇

- **Operação Gradiente**: Aplicado a função escalar – (Leva de escalar para vetor)

$$\nabla f(x,y,z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right)$$

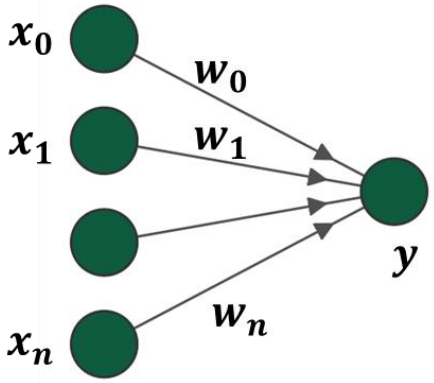
- **Operação Divergente**: Aplicado a um campo vetorial $F = (F_x, F_y, F_z)$ – (Leva de vetor para escalar)

$$\nabla \cdot F = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} + \frac{\partial F_z}{\partial z}$$

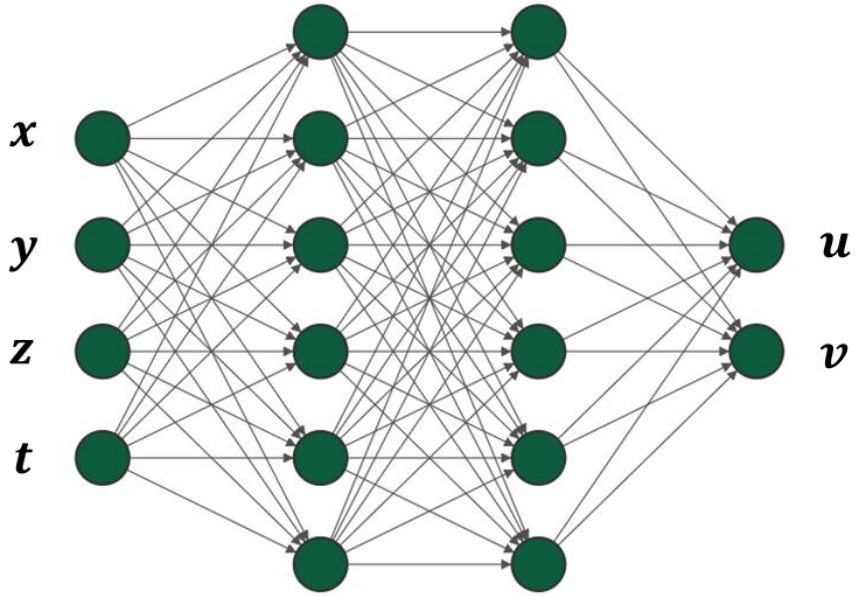
- **Operação Rotacional**: Aplicado a um campo vetorial – (Mantém-se vetor)

$$\nabla \times F = \left(\frac{\partial F_z}{\partial y} - \frac{\partial F_y}{\partial z}, \frac{\partial F_x}{\partial z} - \frac{\partial F_z}{\partial x}, \frac{\partial F_y}{\partial x} - \frac{\partial F_x}{\partial y}\right)$$

Uma breve ideia das NNs - Arquitetura

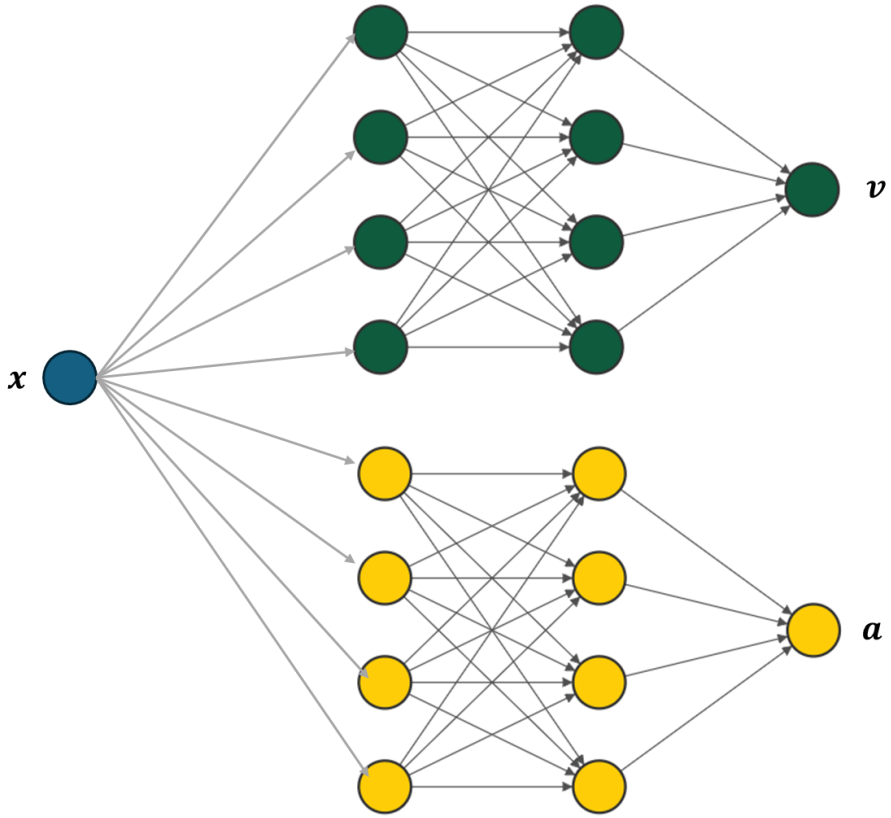


Perceptron



MLP - FNN

Fully-connected neural network

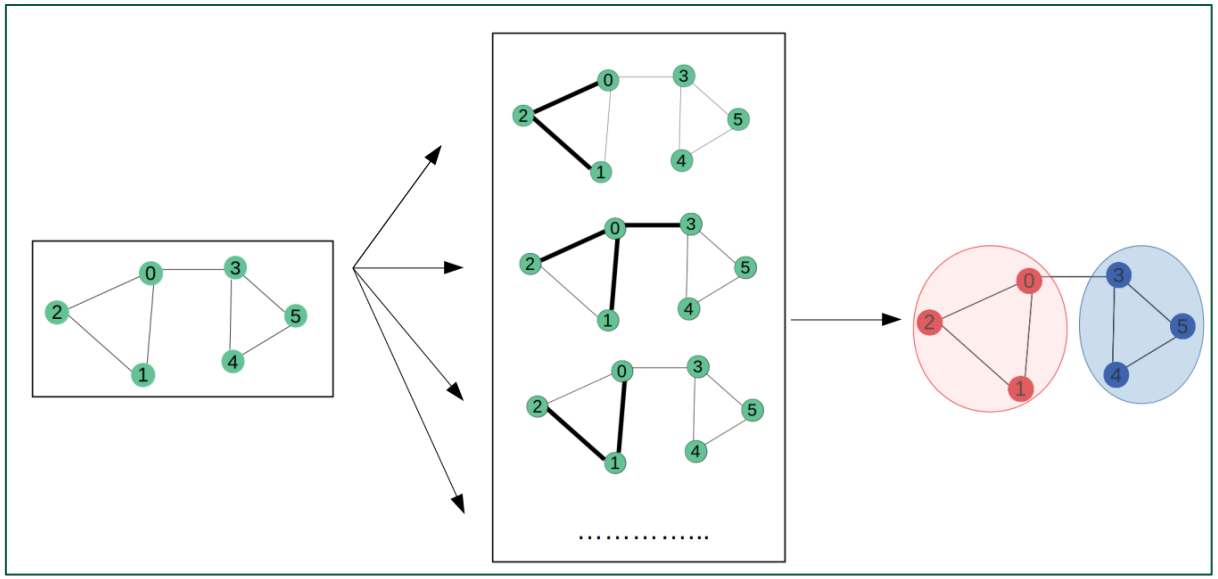


MLP - PFNN

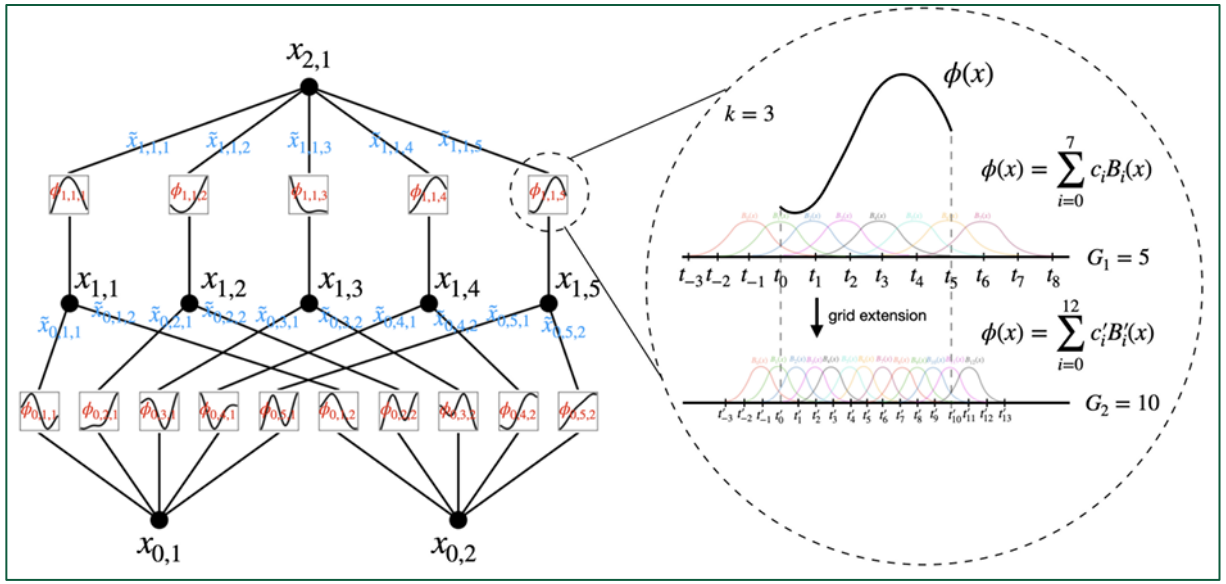
Parallel fully-connected network

MLP: Multi Layer Perceptron

Uma breve ideia das NNs - Arquitetura



Graph Convolution Network



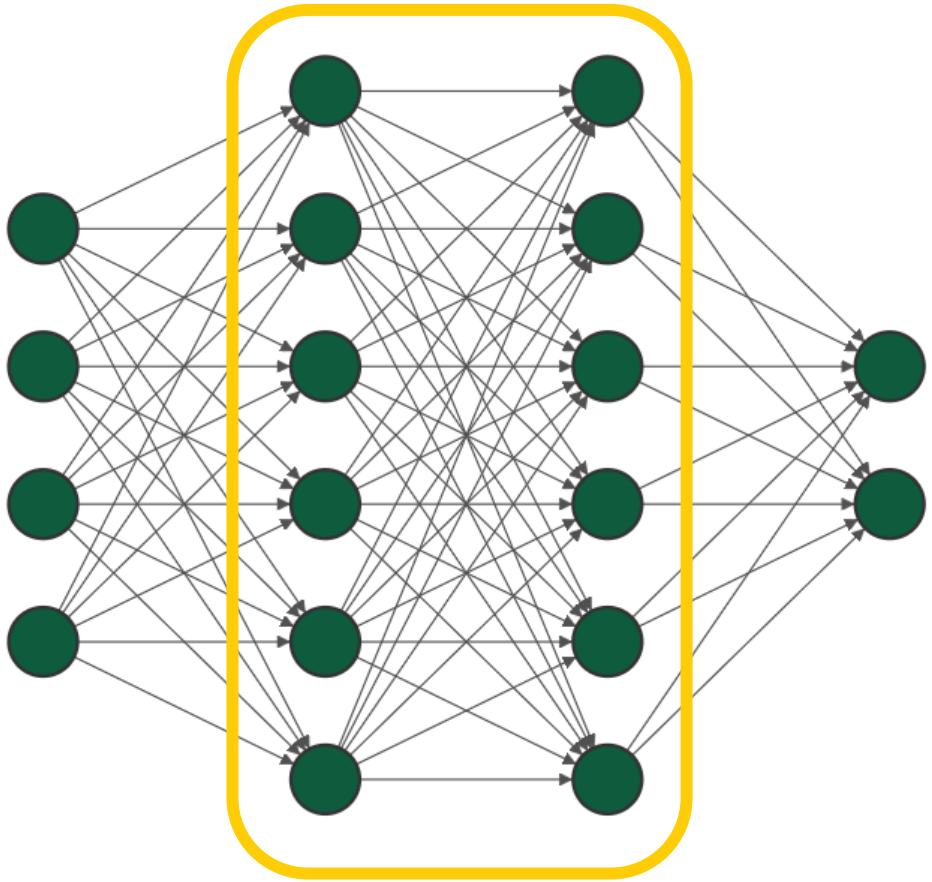
Kolmogorov-Arnold Network

Graph Convolutional Network
https://paperswithcode.com/method/gcn

KAN: Kolmogorov-Arnold Networks
https://arxiv.org/abs/2404.19756

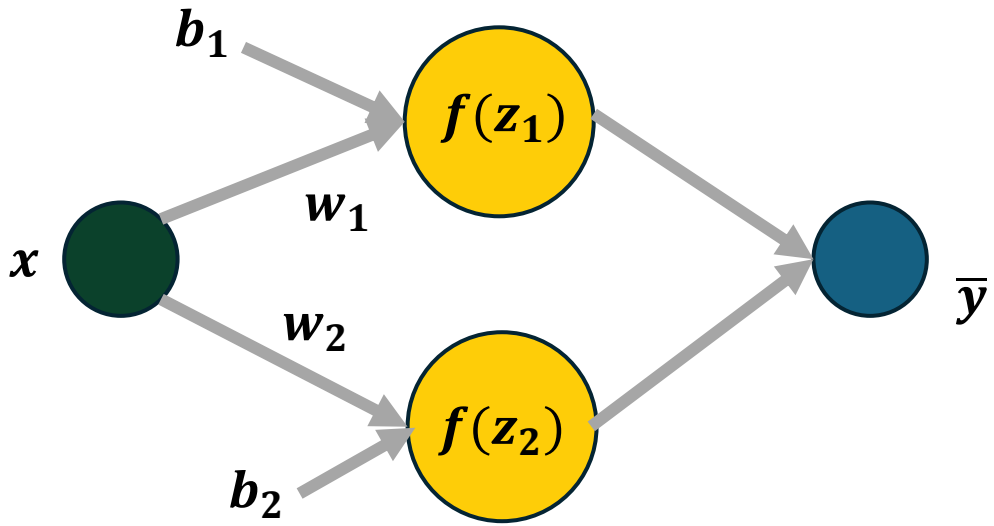
Uma breve ideia das NNs - Parâmetros

Camadas Ocultas



Hidden Layers

Pesos (w) e Viés (b)

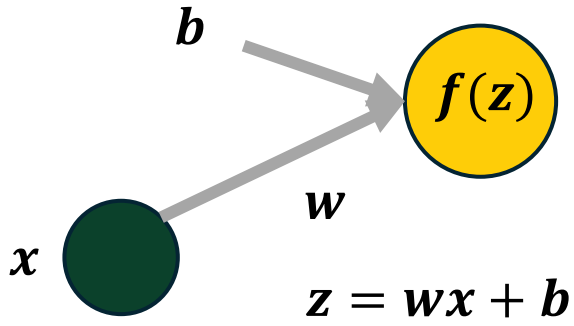


$$z_1 = w_1 x + b_1$$
$$z_2 = w_2 x + b_2$$

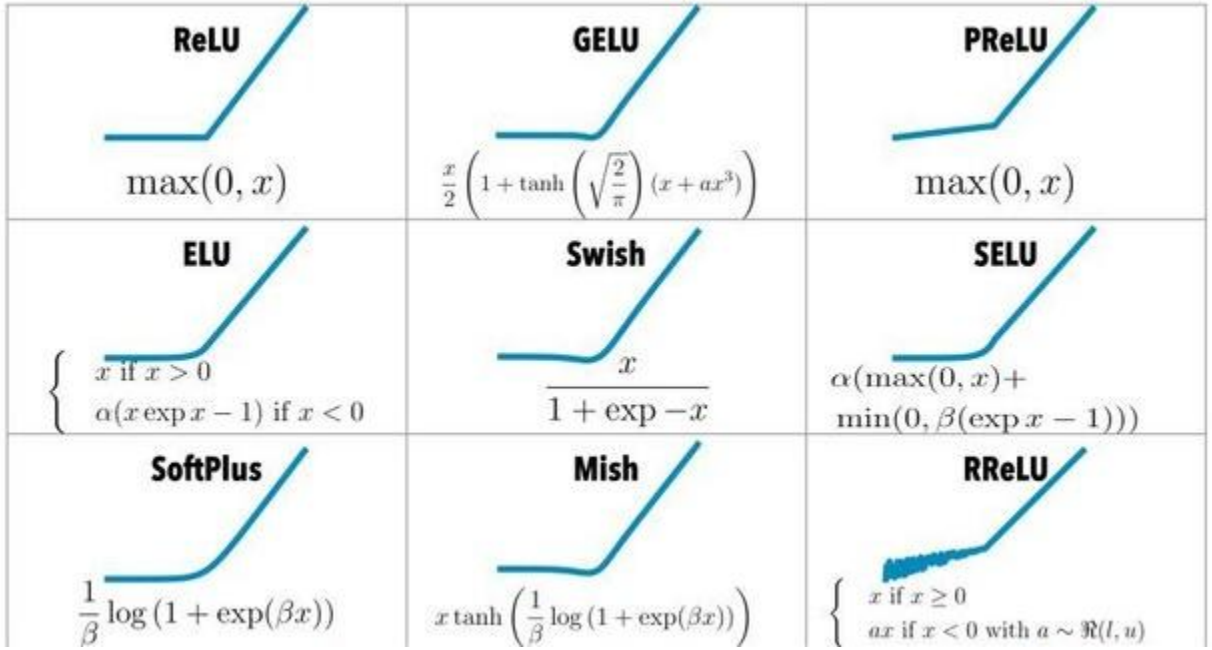
Função de Ativação (f)

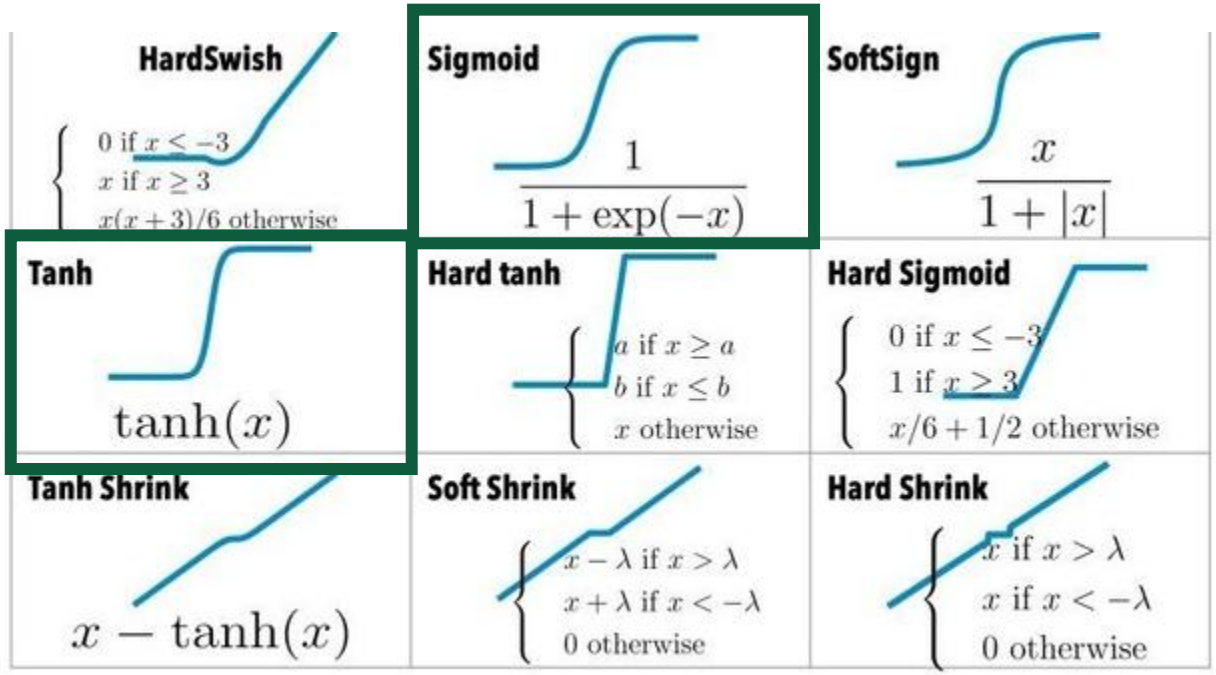
Uma breve ideia das NNs - Parâmetros

Função de Ativação (f)



f
- Sigmoide
- Tangente Hiperbólica





Uma breve ideia das NNs - Parâmetros

Inicializador de Pesos Glorot (Xavier) ou He: A maneira como algoritmo define o intervalo no qual os pesos devem ser iniciados.

Otimizadores Adam ou BFGS: Forma como a rede encontra os melhores pesos e vieses, normalmente utiliza-se **Métodos de Gradiente + *Backpropagation* + Diferenciação Automática**

Épocas: Quantidade de iterações para se obter a convergência dos resultados.

Função de Perda (Loss Function): Função a ser minimizada pelos otimizadores ao longo das épocas, normalmente definida pela função **Mean Square Error (MSE)** ou **Norma L^2**

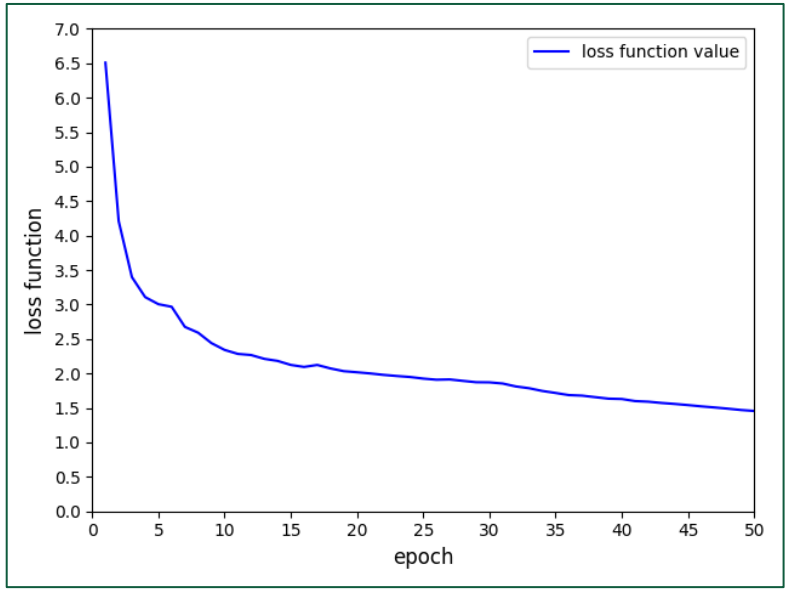
- Xavier Normal Initialization

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

- Xavier Uniform Initialization

$$W \sim U(-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}})$$



$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \qquad |\mathbf{x}| = \sqrt{\sum_{k=1}^{n}|x_k|^2},$$

Uma breve ideia das NNs - Bibliotecas



Sites das Bibliotecas:

- https://numpy.org/
- https://pytorch.org/
- https://www.tensorflow.org/?hl=pt-br
- https://scikit-learn.org/stable/

Os códigos serão desenvolvidos no **Google Colab**:

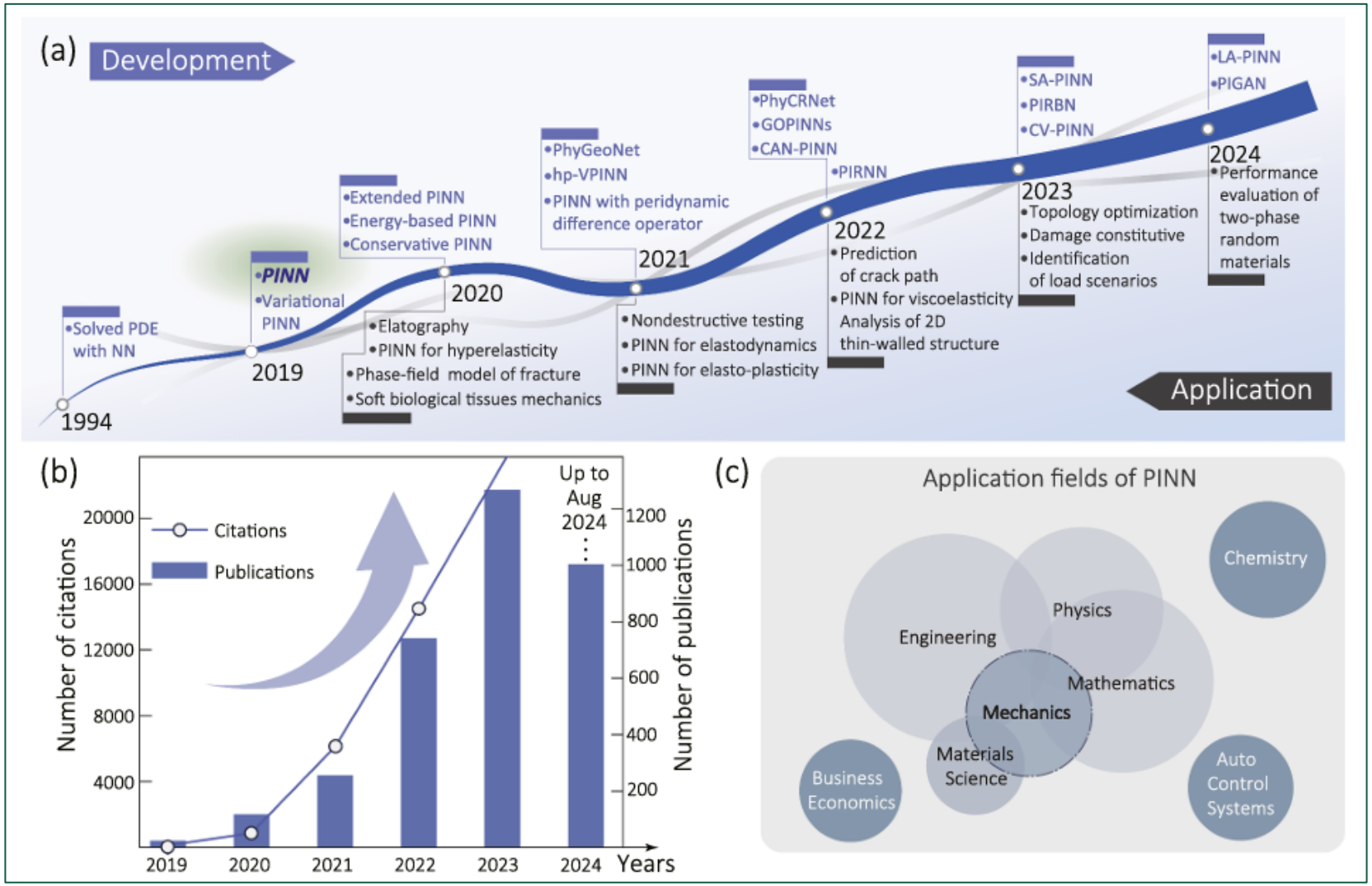
- Basta ter uma conta no GoogleDrive
- Acesso a CPU e GPU
- Não precisa instalar nada na sua máquina

Focaremos em desenvolver os códigos com o **PyTorch**



Redes Neurais e EDs

Redes Neurais e Equações Diferenciais



Hu etal (2024)

Redes Neurais e Equações Diferenciais

Surgimento das estratégias:

Physics Informed Neural Networks (PINN)

- Forma Forte: PINN tradicional
- Forma Energética: Deep Energy Method (DEM)

Operator Learning

- DeepONet
- FNO: Fourier Neural Operator
- PINO: Physics Informed Neural Operator



Physics Informed Neural Networks

O que são PINNs?

Parâmetros, Definições e Particularidades

Desafios e Limitações

Afinal, o que são PINNs?

Considere um **operador** genérico $\mathcal{N}[u(x,t)] = 0$

Uma rede neural tradicional nos dá: $f(x,t,\mathbf{W}) \approx \hat{u}$

Podemos aplicar o mesmo operador sobre a rede

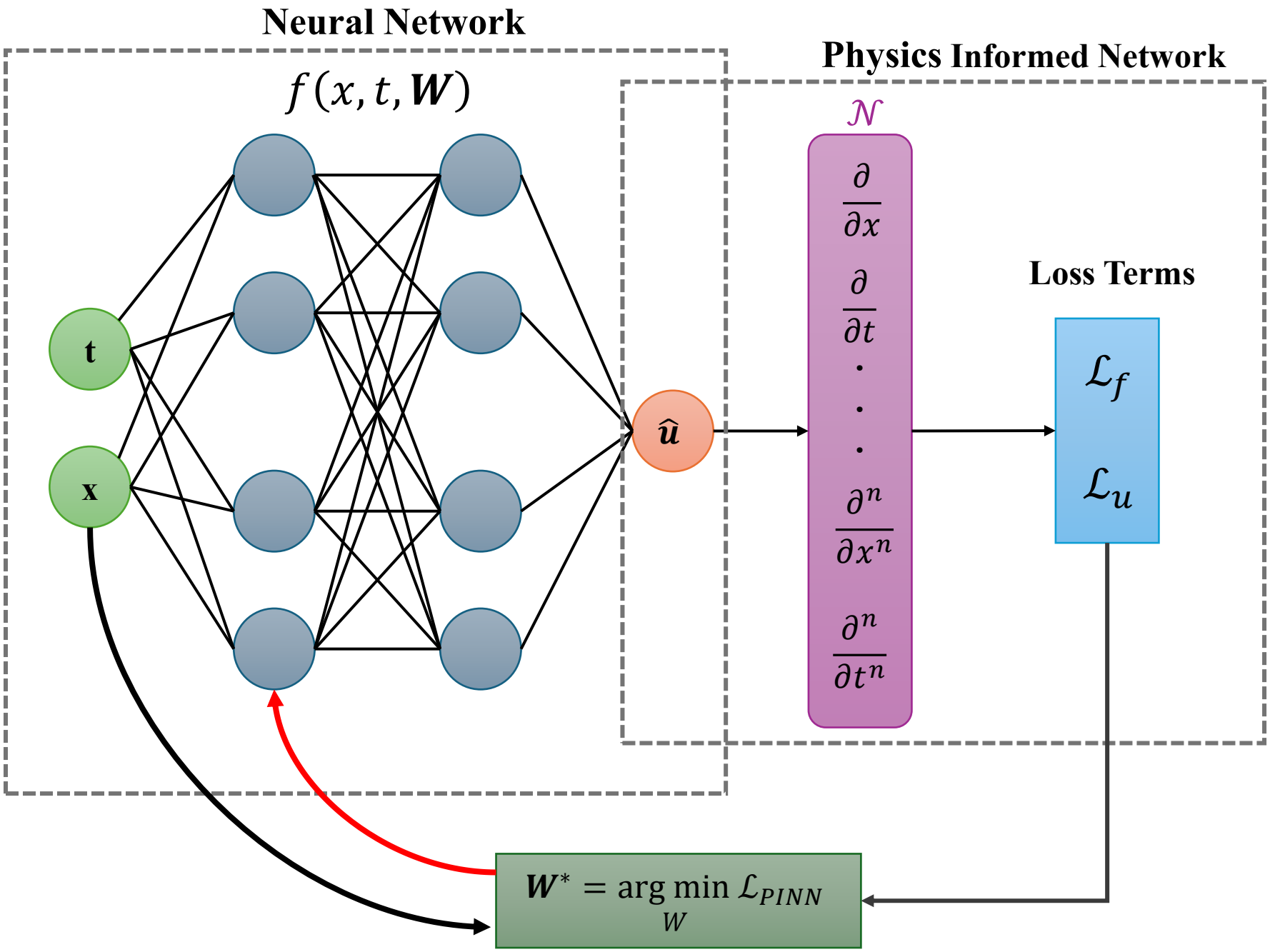
$$\mathcal{N}[\hat{u}] \approx \mathcal{N}[u(x,t)]$$

E atribuir **funções de perda** tanto para esse **operador**, quanto para **condições iniciais** ou de **contorno** do problema

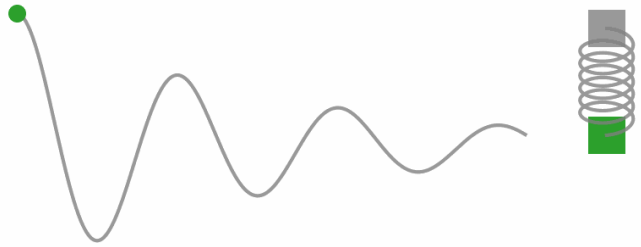
$$\mathcal{L}_f = \frac{1}{N_f}\sum_{i=1}^{N_f} \left|\mathcal{N}(\hat{u}(x_i,t_i))\right|^2, \qquad \mathcal{L}_u = \frac{1}{N_u}\sum_{i=1}^{N_u} \left|u_i - \hat{u}(x_i,t_i)\right|^2,$$

E assim temos uma função de perda total para a PINN, que deve ser minimizada

$$\mathcal{L}_{PINN} = \mathcal{L}_f + \mathcal{L}_u$$


Neural Network
$f(x, t, \boldsymbol{W})$
Physics Informed Network
$\mathcal{N}$
t
x
$\hat{u}$
$\frac{\partial}{\partial x}$
$\frac{\partial}{\partial t}$
$\frac{\partial^n}{\partial x^n}$
$\frac{\partial^n}{\partial t^n}$
Loss Terms
$\mathcal{L}_f$
$\mathcal{L}_u$
$\boldsymbol{W}^* = \arg\min_{W} \mathcal{L}_{PINN}$

Afinal, o que são PINNs?



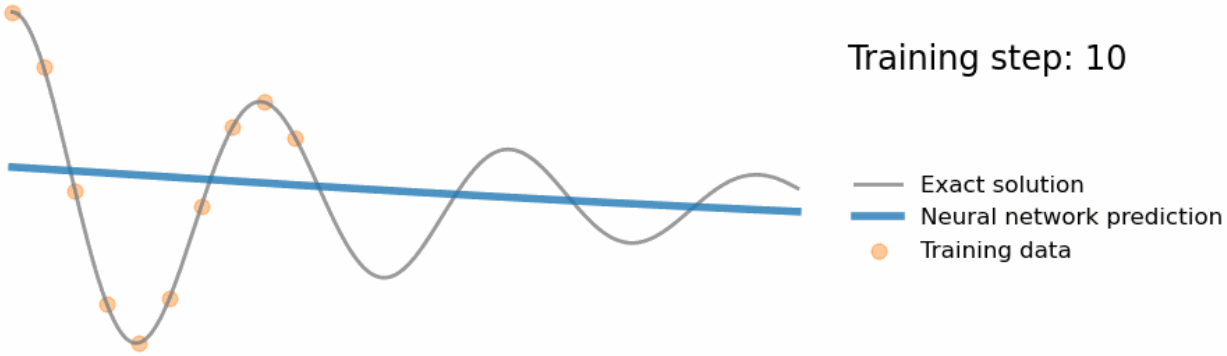
Exemplo com base em [1]

$$M\ddot{u} + C\dot{u} + Ku = 0$$

Rede Neural Tradicional:

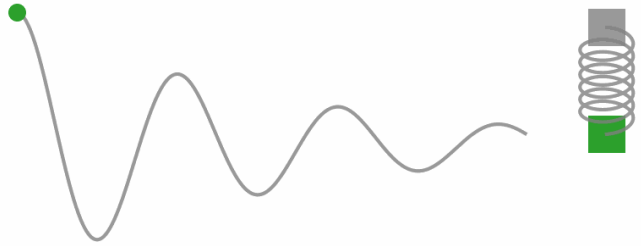
- Fornecer dados da curva original, no qual o ajuste (fitting) é feito somente nesses pontos
- Ter a solução via amostragem, solução analítica ou Runge-Kutta
- Gerar dados de entrada (x) e saída (u)
- A rede será treinada para achar o melhor ajuste para os x_i e $u(x_i)$ **fornecidos** retornando u_{NN}

$$\min \frac{1}{N}\sum_{i}^{N}\left(u_{NN}(x_i) - u(x_i)\right)^2$$



[1] https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network/

Afinal, o que são PINNs?

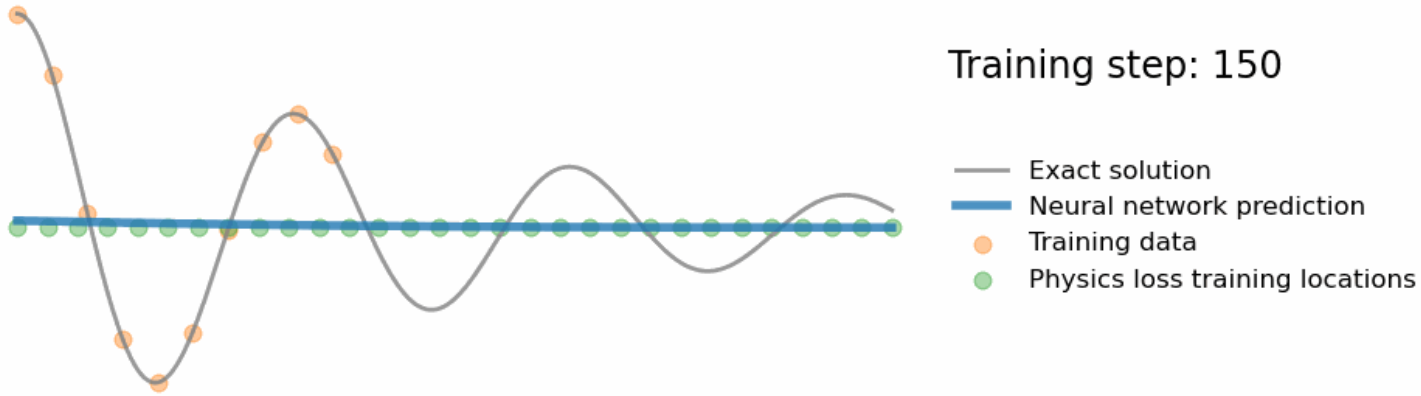


$$M\ddot{u} + C\dot{u} + Ku = 0$$

Redes Informadas por Física

- Fornecer a equação diferencial e calcular em pontos do domínio (x_c)
- Fornecer as condições de contorno: $u(0) = u_0, \;\; \dot{u}(0) = v_0$
- Fornecer dados da curva original (opcional)

$$\min \underbrace{\frac{1}{N_c}\sum_{i}^{N_c}\left(M\ddot{u}_{NN}(x_c) + C\dot{u}_{NN}(x_c) + Ku_{NN}(x_c)\right)^2}_{\mathcal{L}_{PDE}} + \underbrace{\frac{1}{N_{ib}}\sum_{i}^{N_{ib}}\left(u_{NN}(x_{ib}) - u(x_{ib})\right)^2}_{\mathcal{L}_{ICBC}} + \underbrace{\frac{1}{N}\sum_{i}^{N}\left(u_{NN}(x_i) - u(x_i)\right)^2}_{\mathcal{L}_{Dados}}$$

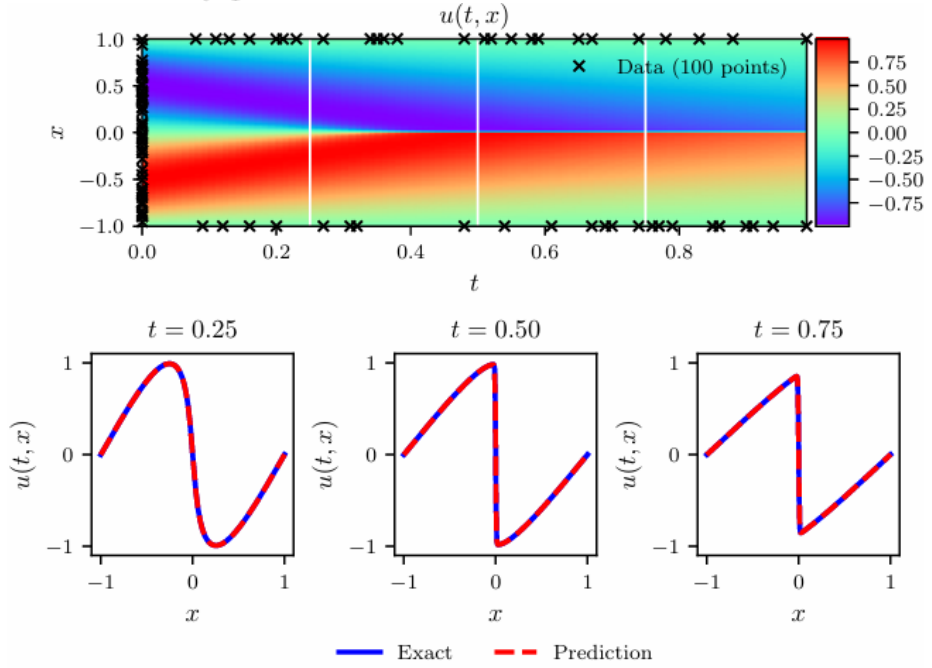


Afinal, o que são PINNs?

Equação de Burgers 1D no espaço

$$u_t + uu_x - (0.01/\pi)u_{xx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1],$$
$$u(0, x) = -\sin(\pi x),$$
$$u(t, -1) = u(t, 1) = 0.$$

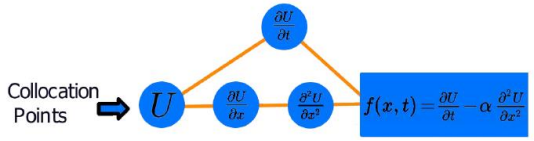
$$MSE_u = \frac{1}{N_u}\sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2, \qquad MSE_f = \frac{1}{N_f}\sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$



Raissi (2017)

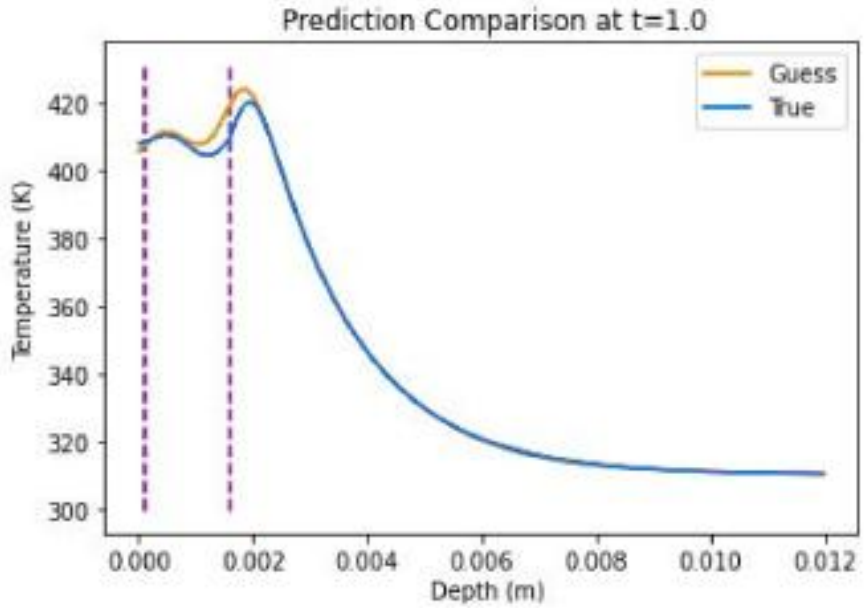
Problema de Transferência de Calor

$$\frac{\partial U}{\partial t} = \alpha \frac{\partial^2 U}{\partial x^2} + Q,$$



$$MSE_U = \frac{1}{N_U}\sum_{n=1}^{N_U} |U^n - U(x_U^n, t_U^n)|^2. \qquad MSE_f = \frac{1}{N_f}\sum_{n=1}^{N_f} |f(x_f^n, t_f^n)|^2.$$

$$\text{Loss} = a_1 \cdot MSE_U + a_2 \cdot MSE_f,$$



Bowman (2023)

Parâmetros, Definições e Particularidades

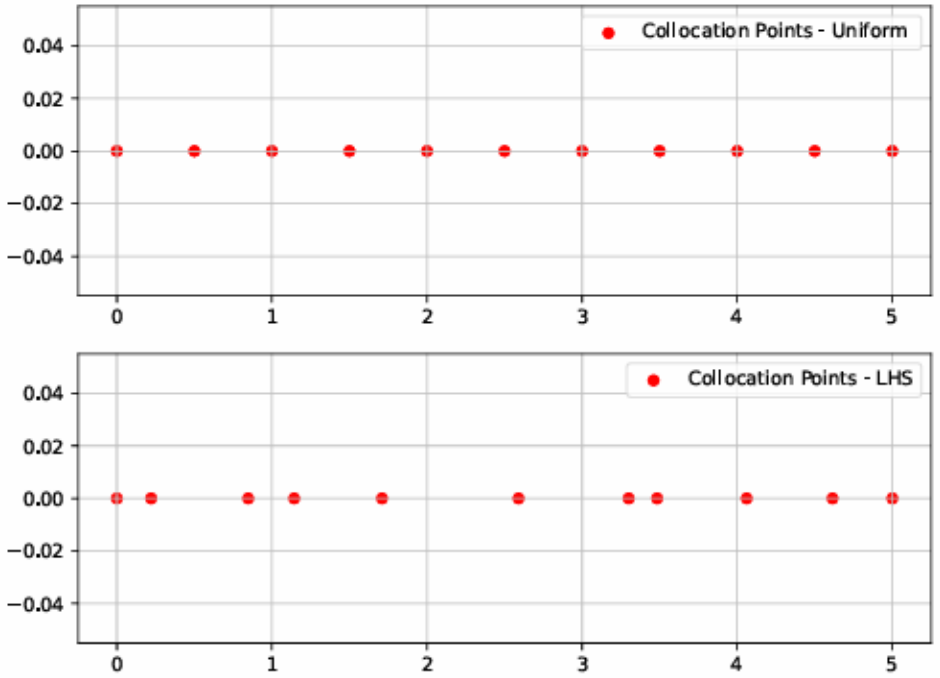
Pontos de colocação: São pontos espalhados pelo domínio e que são introduzidos na Loss durante a minimização.

- Reduzir a quantidade de pontos, reduz o custo computacional;
- **Soluções Triviais:** A PINN retorna um resultado nulo;
- Usar espaçamentos especiais:
  - Distribuição Latin Hypercube Sample
  - Distribuição Aleatória
  - Distribuição de Sobol

Qtd. Camadas Escondidas: 2 à 6 camadas

Qtd. Neurônios: 5 à 64

Função de Ativação: Sigmoide e Tangente Hiperbólica



Paulo Silva (2025) – Não Publicado

Leiteritz (2021)

How to Avoid Trivial Solutions in Physics-Informed Neural Networks

Cuomo (2022)

Scientific Machine Learning Through Physics–Informed Neural Networks: Where we are and What's Next

Vasilly (2024)



Are Two Hidden Layers Still Enough for the Physics–Informed Neural Networks?

Parâmetros, Definições e Particularidades
ICBC* – Fracas (Soft/Weak) e Fortes (Hard/Strong)

Existem duas formas de garantir que as condições de contorno (CCs) sejam atendidas;

Soft

Características

As CCs são adicionadas na Loss
A rede neural é livre porém penalizada
Versão mais utilizada, principalmente quando a imposição é complexa

Vantagens

Fácil Implementar
Flexível para problemas complicados

Desvantagens

Imprecisão
Necessidade de ajustes finos

Hard

Características

As CCs são adicionadas na arquitetura da solução
A rede é construída para satisfazer a CC
Normalmente modifica a solução via funções multiplicadoras

Vantagens

Garante que as CCs sejam satisfeitas
Pode melhorar a precisão e a eficiência do treino

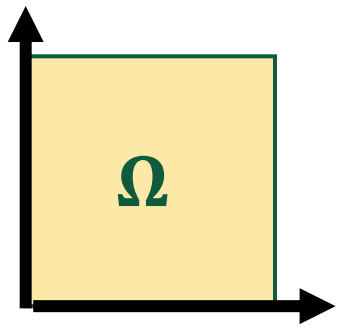
Desvantagens

Mais difícil de implementar
Pode restringir a flexibilidade da rede

*ICBC – Initial Conditions and Boundary Conditions

Parâmetros, Definições e Particularidades

Suporte Compacto: Termo utilizado para **“fechar”** e **“delimitar”** o domínio de estudo, fazendo com que região seja definida em condições que retornam resultados nulos, muito utilizado na definição das **hard constraints.** Veja o exemplo:



$$\Omega = [0\ \ 1]\times[0\ \ 1]$$

Suporte compacto com a função: $x(1-x)$

***Batch size* (Tamanho do Lote):** define quantos exemplos do conjunto de dados são processados antes de atualizar os pesos do modelo. O *batch size* determina:

- Quantos dados são usados por vez para calcular os gradientes.
- Com que frequência os pesos do modelo são atualizados.
- **Valores comuns:** 16, 32, 64, 128

Desafios e Limitações

Apesar de promissoras, as PINNs ainda enfrentam dificuldades técnicas, como:

- **Viés espectral:** redes tendem a aprender primeiro as frequências baixas.
- **Dificuldade de treinamento:** as funções de perda podem ser mal condicionadas.
- Sensibilidade à escolha dos pesos entre os termos da perda (dados vs. física).
- **Escalabilidade** para problemas muito grandes ou com soluções de comportamento local complexo.



Bibliotecas

Bibliotecas de PINN

DeepXDE

Bibliotecas de PINN

Name	Address of the open-source code	Language	Refs.
NeuralPDE	https://github.com/SciML/NeuralPDE.jl.git	Julia	[129]
Neural Solvers	https://github.com/Photon-AI-Research/NeuralSolvers.git	Python	[130]
IDRLnet	https://github.com/idrl-lab/idrlnet.git	Python	[131]
PINA	https://github.com/mathLab/PINA.git	Python	[132]
TensorDiffEq	https://github.com/SciML/TensorFlowDiffEq.jl.git	Julia	[133]
DeepINN	https://github.com/praksharma/DeepINN.git	Jupyter Notebook	[134]
NeuralUQ	https://github.com/Crunch -UQ4MI/neuraluq.git	Python	[135]
Neurodiffeq	https://github.com/NeuroDiffGym/neurodiffeq.git	Python	[136]
DeepXDE	https://github.com/lululxvi/deepxde	Python	[137]
SciANN	https://github.com/sciann/sciann.git	Python	[138]
SimNet	https://developer.nvidia.com/simnet	Python	[139]
ADCME	https://github.com/kailaix/ADCME.jl.git	Julia	[140]
PDEBench	https://github.com/pdebench/PDEBench.git	Jupyter Notebook	[141]
PINNs-Torch	https://github.com/rezaakb/pinns-torch.git	Python	[142]
Neural Eikonal solver	https://github.com/sgrubas/NES.git	Jupyter Notebook	[143]
PinnDE	https://github.com/JB55Matthews/PinnDE	Python	[144]
MLniCS	https://github.com/MLniCS/MLniCS	Jupyter Notebook	[145]
PINNLib	https://github.com/m-raj/PINNLib	Python	[146]
Porch	https://github.com/leiterrl/porch	Python	
Jinns	https://github.com/mia-jinns/jinns	Jupyter Notebook	

Hu etal (2024)

DeepXDE

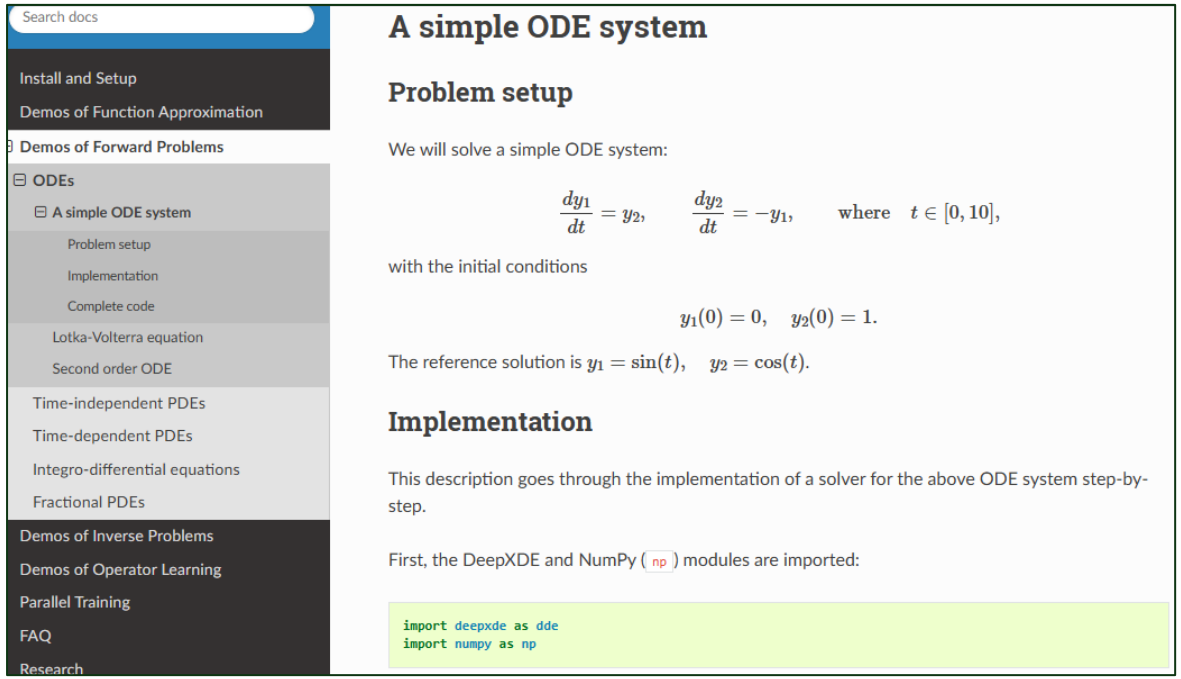
DEEPXDE: A DEEP LEARNING LIBRARY FOR SOLVING DIFFERENTIAL EQUATIONS

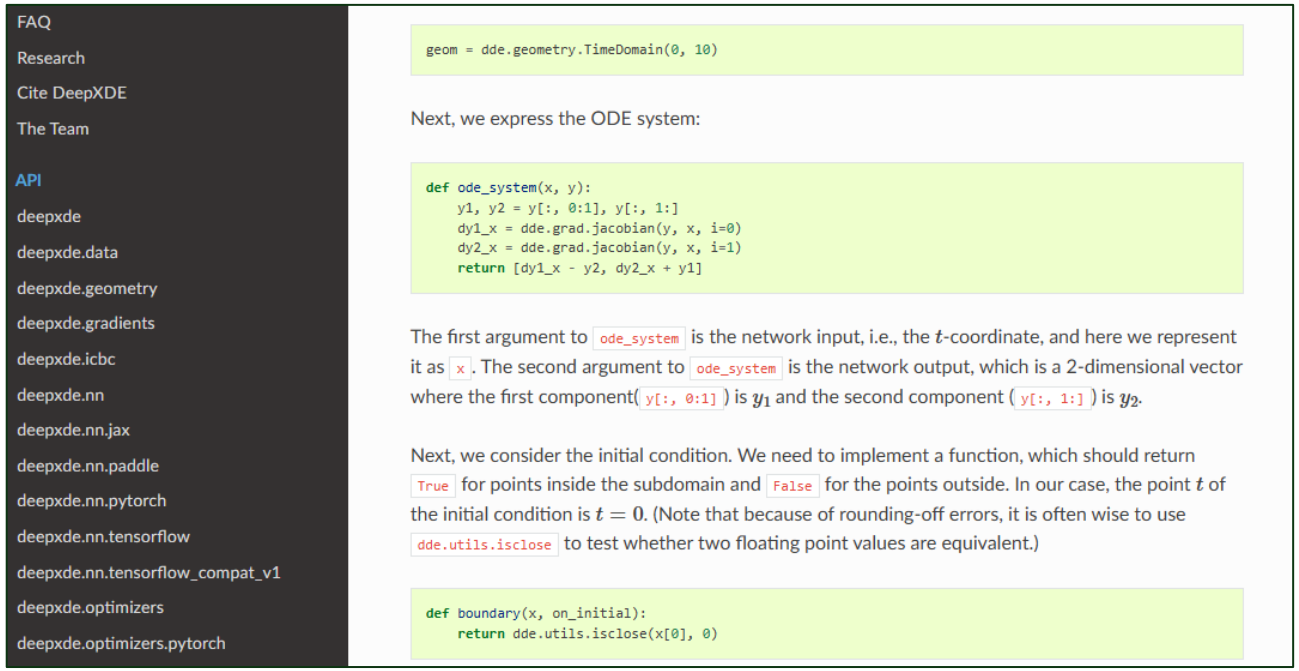
LU LU*, XUHUI MENG*, ZHIPING MAO*, AND GEORGE EM KARNIADAKIS*†

https://doi.org/10.48550/arXiv.1907.04502

Múltiplos backends: PyTorch, TensorFlow, JAX e PaddlePaddle.

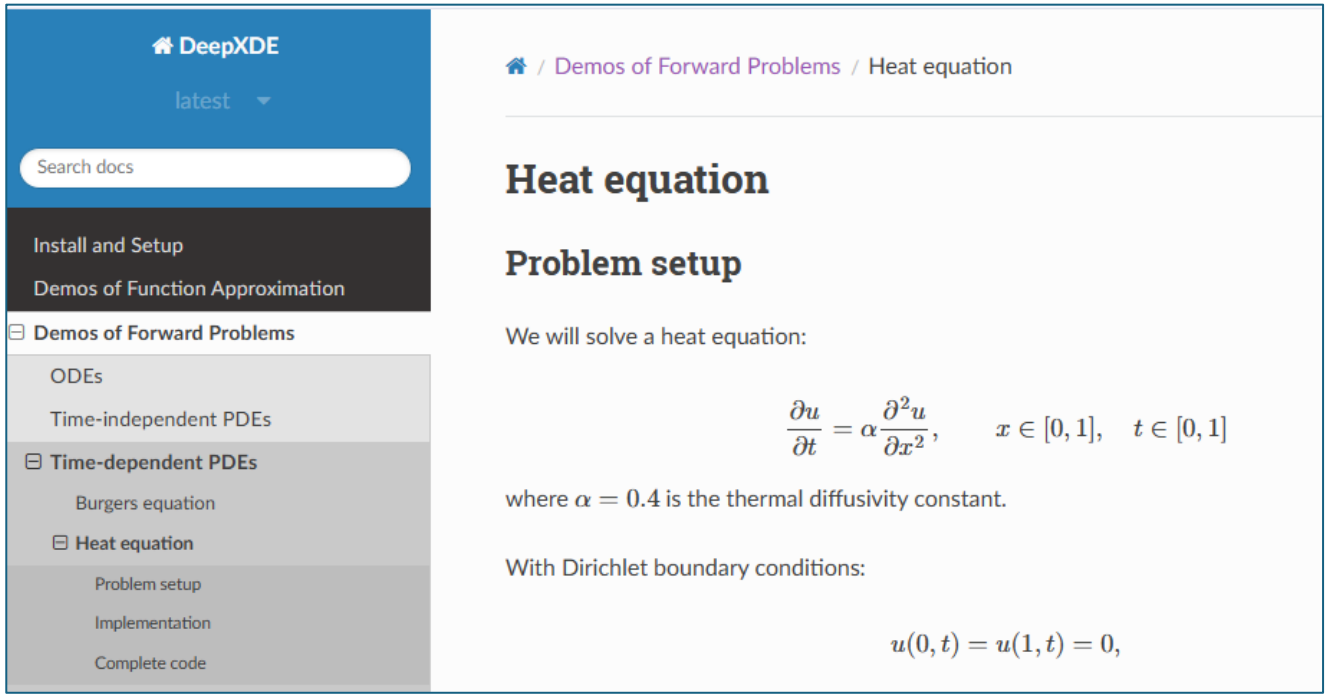
https://deepxde.readthedocs.io/en/latest/index.html



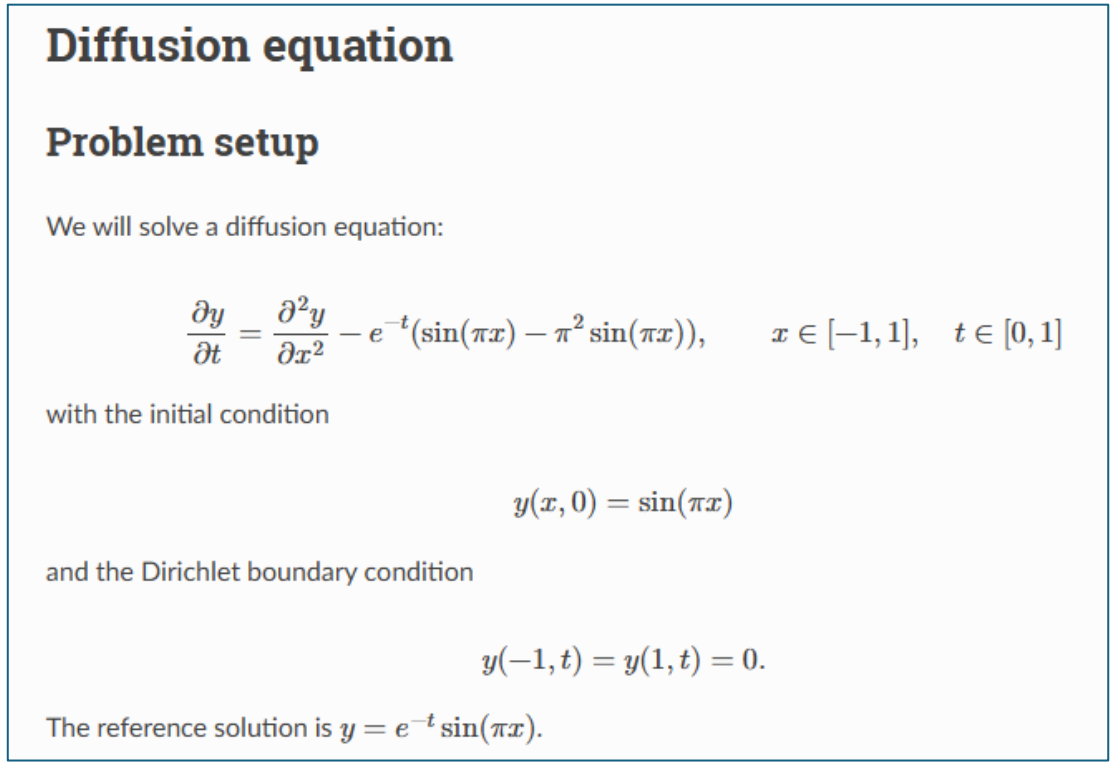


YouTube: https://youtube.com/playlist?list=PLM7DTyYjZGuLmg3f6j40fEF18jyQmYsC2

DeepXDE problemas de Transcal



https://deepxde.readthedocs.io/en/latest/demos/pinn_forward/heat.html



https://deepxde.readthedocs.io/en/latest/demos/pinn_forward/diffusion.1d.html



Aplicações em Transferência de Calor

Frentes da Transferência de Calor

Problemas Diretos e Inversos

Frentes de Transferência de Calor

Grandes Áreas

Condução de calor

Convecção de calor

Camadas-limite térmicas

Transferência de calor transiente

Transferência de calor conjugada

Foco de Determinação

Temperatura

Fluxo de calor

Gradientes térmicos

Propriedades térmicas

Coeficientes de transferência

Fontes de calor

Temas

Problemas Físicos

Paredes planas

Geração interna de energia

Paredes compostas

Condução bidimensional

Capacitância global

Camada-limite hidrodinâmica

Camada-limite térmica

Identificações

Condutividade térmica (k)

Difusividade térmica (α)

Coeficiente convectivo (h)

Geração volumétrica ($\dot{q}$)

Condições de contorno

Problemas Diretos

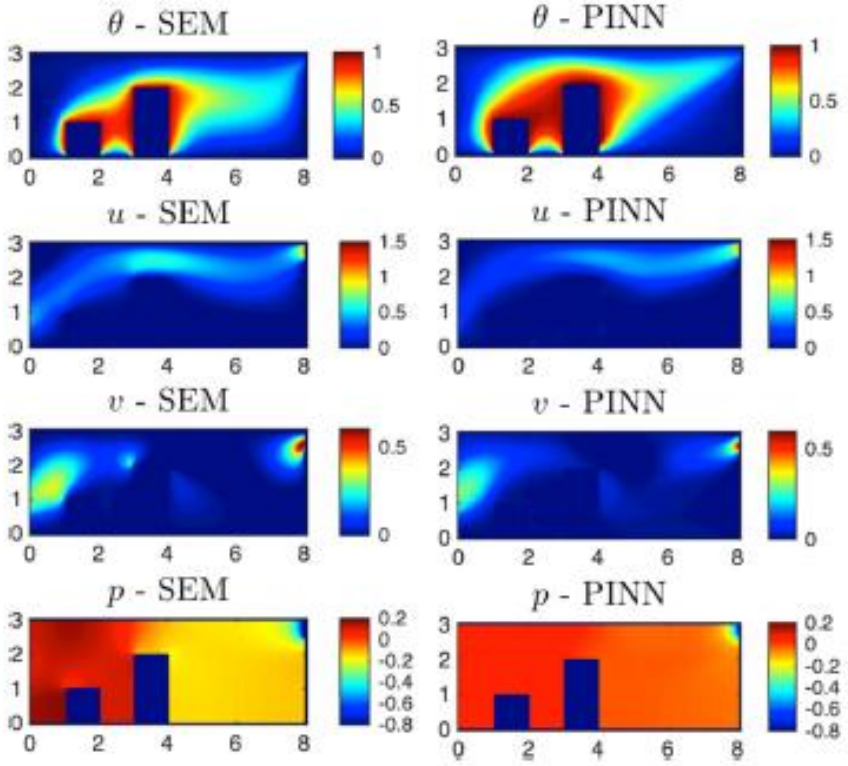
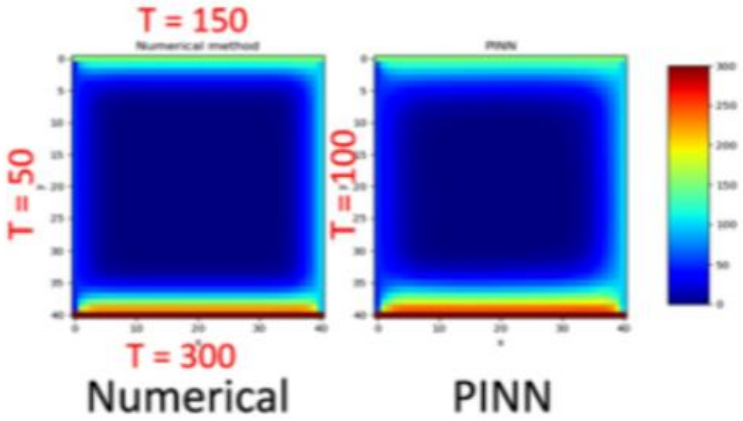
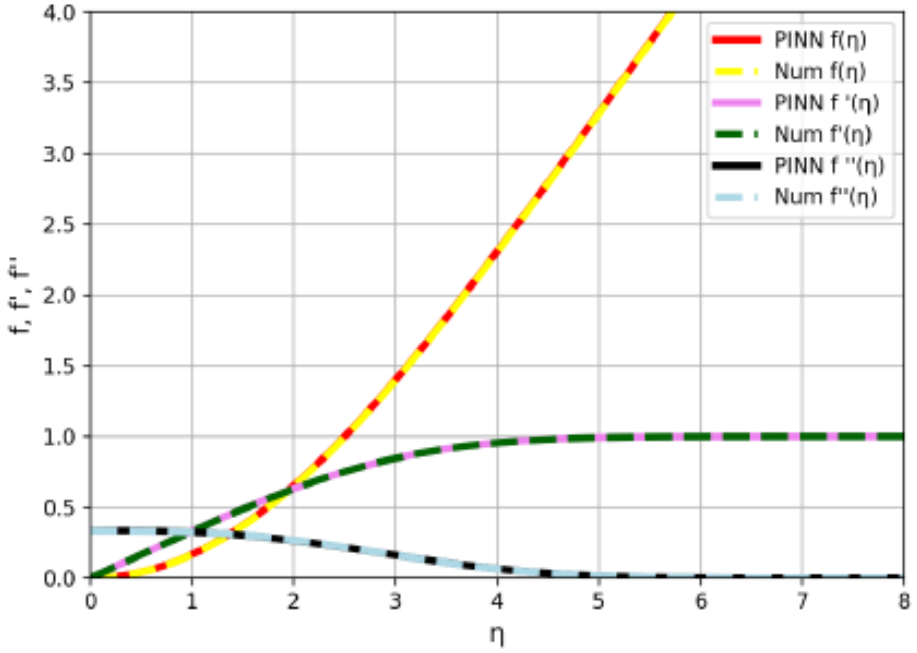


Fig. 3 Forced convection in an enclosure: comparison of θ, u, v, and p inferred by PINN and the reference simulation results by the spectral element method

Convecção

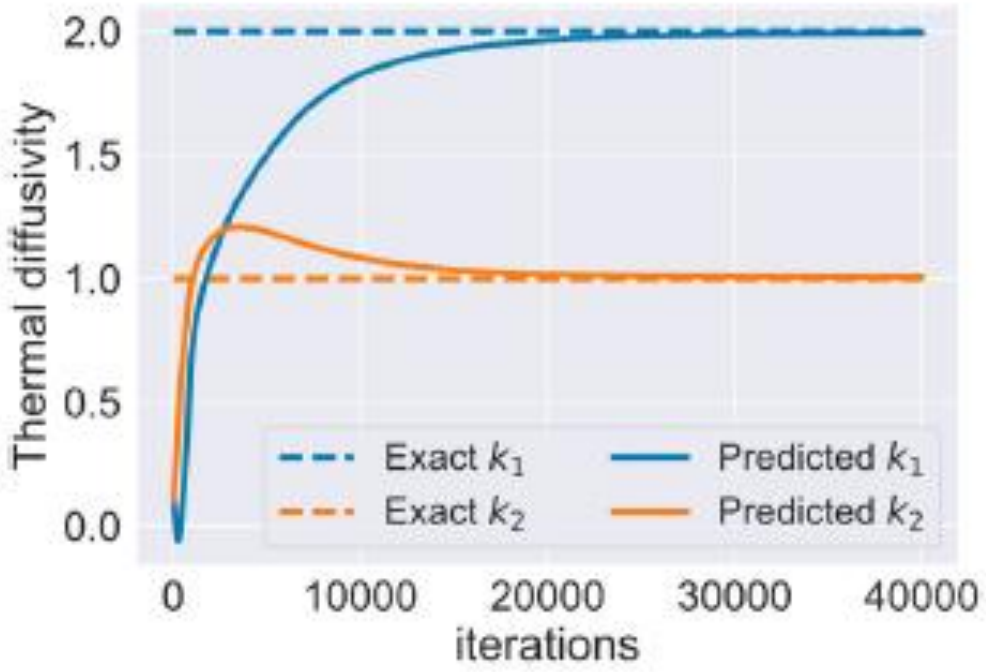


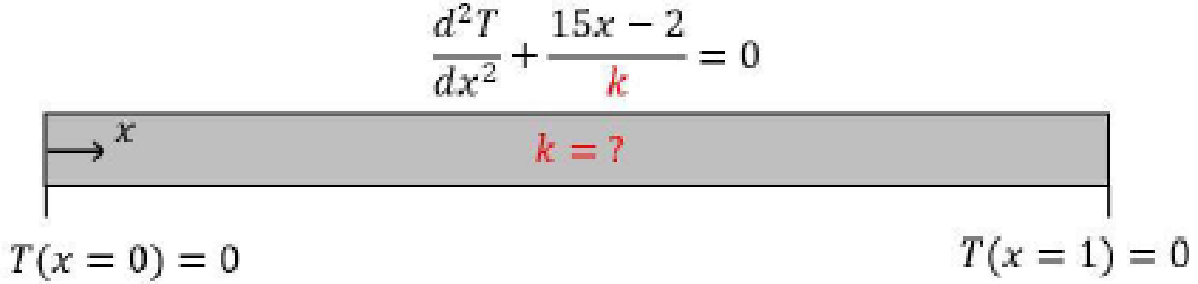
Condução



Solução de Blasius

Problemas Inversos





Identificação de Parâmetros



Vídeos e Referências

Vídeos Recomendados

Vídeos

Physics Informed Neural Networks (PINNs): "PyTorch" Solve Physical Systems with Deep Neural Networks: https://youtu.be/MtHvs5FvtME?si=MeGVe9HF8pCkKN2j <Acesso em: 13/05/2025>

Physics Informed Neural Networks for Fluid Mechanics: https://youtu.be/EHrgSPHZG3Y?si=JUOWe1pC_7bt6i18 <Acesso em: 13/05/2025>

Playlists

Neural Networks / Deep Learning: https://youtube.com/playlist?list=PLblh5JKOoLUIxGDQs4LFFD--41Vzf-ME1 <Acesso em: 03/06/2025>

Physics Informed Machine Learning: https://youtube.com/playlist?list=PLMrJAkhIeNNQ0BaKuBKY43k4xMo6NSbBa <Acesso em: 06/06/2025>

Physics Informed Neural Networks (PINNs): https://youtube.com/playlist?list=PLM7DTyYjZGuLmg3f6j40fEF18jyQmYsC2 <Acesso em: 06/06/2025>

Referências

Bowman (2023): Physics-Informed Neural Networks for the Heat Equation with Source Term under Various Boundary Conditions

Cai etal (2021): Physics-Informed Neural Networks for Heat Transfer Problems

Gao etal (2022): Physics-informed graph neural Galerkin networks: A unified framework for solving PDE-governed forward and inverse problems

Hu etal (2024): Physics-informed Neural Networks (PINN) for computational solid mechanics: Numerical frameworks and applications

Jagtap (2020): Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems

Raissi (2017) - Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Di erential Equations

Wang etal (2024a)- Artificial intelligence for partial differential equations in computational mechanics: A review



Obrigado!